



UNIVERSIDADE D
COIMBRA

QCML2- Image classification with Hybrid Quantum Convolutional Neural Networks

Results

Members:

Miguel Castela 2022212972

Supervised by Professor Doctor Jorge Lobo

March, 2025

Table of Contents

Introduction	2
First evaluations with the Qiskit tutorials	5
Diferent tipos of QNN	5
Conclusions and future work	5

Introduction

Artificial neural networks are currently one of the most popular models used in machine learning. A subcategory of these called convolutional neural networks (CNN) is particularly popular. These are networks mainly used for the task of image recognition. Using pooling and convolution layers they extract structured information about images. This project will be related to CNN topic but also to Quantum computing. Quantum machine learning leverages the principles of quantum mechanics to process information, allowing for the creation of algorithms that can potentially outperform classical counterparts. By harnessing quantum bits or qubits, these algorithms can explore multiple possibilities simultaneously, offering a unique approach to solving complex problems. While still in the early stages of development, quantum machine learning holds promise for revolutionizing certain computational tasks, particularly those involving large-scale optimization and data analysis.

The first step of this project was using the Qiskit framework, combined with pytorch to construct a HQNN. After studying the first three qiskit tutorials ("Quantum Neural Networks", "Neural Network Classifier" & "Training a Quantum Model on a Real Dataset") i began to implement the first version of the QNN. This was done by implementing the "Torch Connector and Hybrid QNNs" tutorial:

- The first step is to initialize the Estimator. The Estimator is a class in Qiskit that allows you to estimate the expectation values of quantum circuits. It is used to evaluate the performance of quantum models by measuring their outputs.

```
estimator = Estimator()
```

- I then, using torchvision, load the MNIST training dataset(The MNIST database is a large database of handwritten digits) and apply a transformation to convert images to tensors.

A tensor is a generalization of vectors and matrices to any number of dimensions. They are the fundamental data structure in PyTorch, similar to NumPy arrays but with additional capabilities for GPU acceleration.

```
X_train = datasets.MNIST(
    root="./data", train=True, download=True,
    transform=transforms.Compose([transforms.ToTensor()])
)
```

- I filter the dataset to keep only digits labeled 0 and 1 (first `n_samples` of each)
I use the `np.append` function to concatenate the indices of the two classes (0 and 1) and then filter the dataset using these indices. The `np.where` function returns the indices of elements in an array that satisfy a given condition.
The `X_train.data` and `X_train.targets` attributes are then updated to include only the selected samples.
The `X_train.data` attribute contains the image data, while `X_train.targets` contains the corresponding labels.

```

idx = np.append(
    np.where(X_train.targets == 0)[0][:n_samples]
    , np.where(X_train.targets == 1)[0][:n_samples]
)
X_train.data = X_train.data[idx]
X_train.targets = X_train.targets[idx]

```

- I define the PyTorch DataLoader for the filtered training data, this is used to load the data in batches during training. The `shuffle=True` argument ensures that the data is shuffled before each epoch, which helps improve the model's generalization.

```
train_loader = DataLoader(X_train, batch_size=batch_size, shuffle=True)
```

- I load the MNIST dataset again to create the test set, this time with `train=False` to indicate that we want the test set. The same transformation is applied as before. I again filter the test dataset to keep only digits labeled 0 and 1 (first `n_samples` of each) and define the DataLoader for the filtered test data.

```

n_samples = 50
X_test = datasets.MNIST(
    root="./data", train=False, download=True,
    transform=transforms.Compose([transforms.ToTensor()])
)
...

```

- Then I create the quantum neural network. At this point, a key decision must be made: **how to encode the classical input features into a quantum state** — a process known as *feature mapping* or *data encoding*.

Qiskit provides several options for this, the most common being:

- **ZFeatureMap**

A simple quantum feature map where each input value is encoded as a rotation on a single qubit using Rz gates. This approach does **not include entanglement** and is typically used for simpler datasets or models with lower computational complexity.

- **ZZFeatureMap**

A more expressive feature map that includes both single-qubit rotations and **entangling gates (ZZ interactions)** between qubits. This allows the model to capture complex relationships between features, making it more suitable for richer or more structured datasets.

These feature maps are not just about storing the features — they are essential for **embedding classical data (such as pixel values from images)** into the quantum circuit so that the quantum neural network can process it.

Other alternatives to these include:

- **PauliFeatureMap** – a generalized feature map that supports different combinations of Pauli operators.
- **Custom feature maps** – where users design their own encoding strategies tailored to specific datasets or problems.

The choice of feature map directly influences the expressiveness and performance of the quantum model. In this case, i used the **ZZFeatureMap**.

The next choice is the **ansatz** (the quantum circuit structure). The ansatz defines the architecture of the quantum neural network, including the number of qubits, layers, and gates used. Qiskit provides several predefined ansatz classes, such as:

- **EfficientSU2**
A popular ansatz that uses a combination of single-qubit rotations and entangling gates. It is designed to be efficient in terms of circuit depth and is suitable for a wide range of problems.
- **RealAmplitudes**
This ansatz uses real-valued parameters and is often used for variational algorithms. It can be more expressive than EfficientSU2 but may require more qubits and gates.
- **TwoLocal**
A flexible ansatz that allows users to specify the types of gates and the connectivity between qubits. It can be tailored to specific problems but may require more tuning.

The choice of ansatz is crucial as it determines the expressiveness and complexity of the quantum model. In this case i used the RealAmplitudes.

```
def create_qnn():
    feature_map = ZZFeatureMap(2)
    ansatz = RealAmplitudes(2, reps=1)
    qc = QuantumCircuit(2)
    qc.compose(feature_map, inplace=True)
    qc.compose(ansatz, inplace=True)

    # REMEMBER TO SET input_gradients=True FOR ENABLING HYBRID GRADIENT BACKPROP
    qnn = EstimatorQNN(
        circuit=qc,
        input_params=feature_map.parameters,
        weight_params=ansatz.parameters,
        input_gradients=True,
        estimator=estimator,
    )
    return qnn

qnn4 = create_qnn()
```

- The next step is to define the neural network with a quantum layer.
-

First evaluations with the Qiskit tutorials

Diferent tipos of QNN

Conclusions and future work